Lecture 11: The Deep Learning Revolution

Modern CNN Architectures: From AlexNet to ResNet and Beyond

CSE 329: Machine Learning — July 2025

"The history of deep learning is a story of going deeper—and learning how to do it."

Learning Objectives: By the end of this lecture, you will be able to:

1. Trace the evolution of CNN architectures and the problems each solved
2. Explain how Batch Normalization enables training of deep networks
3. Understand the purpose of 1×1 convolutions and Inception modules
4. Derive why skip connections solve the degradation problem in ResNet
5. Apply design principles to construct your own CNN architectures

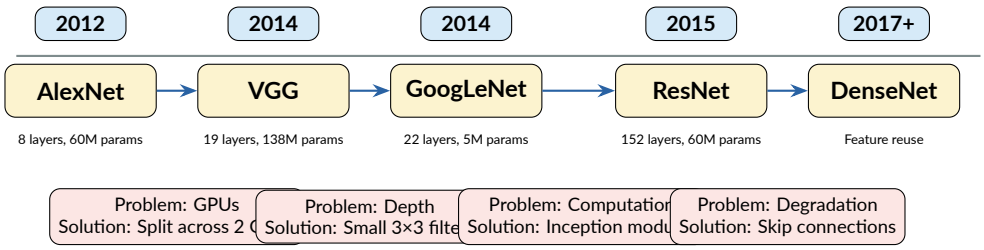
---

1 The Quest for Depth: A Historical Narrative

Think About It

In 2012, a neural network won ImageNet by a **huge margin**. By 2015, networks had grown from 8 to 152 layers. What problems did researchers face going deeper, and how did they solve them?

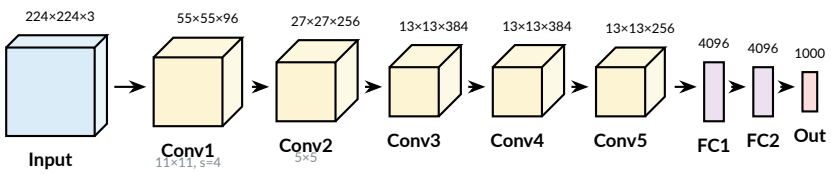
The story of modern CNNs is one of **progressive problem-solving**:



2 AlexNet (2012): The Breakthrough

AlexNet won ImageNet 2012 with **16.4% top-5 error** (vs. 26.2% for the runner-up)—the moment deep learning became mainstream.

2.1 Architecture Diagram



MaxPool after Conv1, Conv2, Conv5 | ReLU after each layer | Dropout in FC layers

2.2 Key Innovations

- **ReLU activation**: $f(x) = \max(0, x)$ — 6× faster training than tanh
- **Dropout** (p=0.5): Regularization for FC layers to prevent overfitting
- **Data augmentation**: Random crops, horizontal flips, color jittering
- **GPU training**: Model split across 2 GTX 580 GPUs (3GB each)
- **Local Response Normalization**: Normalize across channels (later replaced by BatchNorm)

3 VGG (2014): The Power of Small Filters

Think About It

Which is better: one 5×5 convolution or two stacked 3×3 convolutions? They have the same receptive field, but very different properties...

3.1 The Key Insight: Stacking Small Filters

VGG's Design Principle: Replace large filters with stacks of 3×3 filters.
Receptive field equivalence:

- One 5×5 filter ≡ Two 3×3 filters (both see 5×5 region)
- One 7×7 filter ≡ Three 3×3 filters (both see 7×7 region)

Why is stacking 3×3 better?

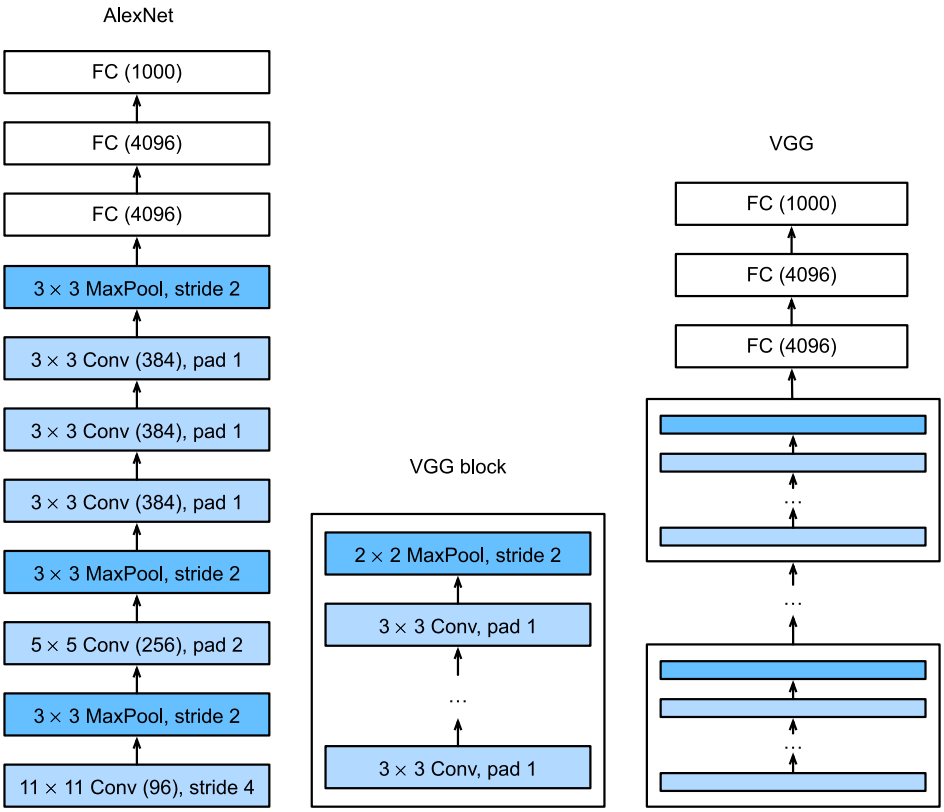
Metric	One 5×5	Two 3×3 stacked
Receptive field	5×5	5×5
Parameters (C channels)	$25C^2$	$2 \times 9C^2 = 18C^2$
Non-linearities	1 ReLU	2 ReLUs

Key Idea

Stacking small filters gives: (1) **fewer parameters** (28% reduction), and (2) **more non-linearity** (more expressive). This is why modern CNNs almost exclusively use 3×3 filters.

3.2 VGG Architecture Pattern

VGG uses a simple, uniform structure: `[Conv 3×3] × n → MaxPool 2×2` repeated, doubling channels each block.



Stage	Output Size	VGG-16 Layers
Input	$224 \times 224 \times 3$	–
Block 1	$112 \times 112 \times 64$	Conv3-64 × 2, MaxPool
Block 2	$56 \times 56 \times 128$	Conv3-128 × 2, MaxPool
Block 3	$28 \times 28 \times 256$	Conv3-256 × 3, MaxPool
Block 4	$14 \times 14 \times 512$	Conv3-512 × 3, MaxPool
Block 5	$7 \times 7 \times 512$	Conv3-512 × 3, MaxPool
FC	1000	FC-4096, FC-4096, FC-1000

Warning

VGG-16 has **138 million parameters**, with 90% in FC layers! The FC layers alone need 102M parameters. This motivated the search for more efficient designs.

4 Batch Normalization: The Training Stabilizer

Think About It

Deep networks suffer from *internal covariate shift*—the distribution of layer inputs changes during training. How can we stabilize training as we go deeper?

4.1 The Problem: Shifting Distributions

As weights update, the distribution of inputs to each layer changes, forcing later layers to constantly adapt. This slows training and requires careful initialization.

4.2 The Solution: Normalize Each Layer

Batch Normalization (for a mini-batch $\mathcal{B} = \{x_1, ..., x_m\}$):
Step 1: Compute batch statistics

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$

Step 2: Normalize

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Step 3: Scale and shift (learnable parameters γ, β)

$$y_i = \gamma \hat{x}_i + \beta$$

Why the learnable γ and β? Pure normalization might hurt expressivity (e.g., forcing sigmoid inputs to linear region). The network can learn to undo normalization if needed: setting $\gamma = \sigma$ and $\beta = \mu$ recovers the original.

4.3 BatchNorm Benefits

Key Idea

Batch Normalization enables:

- **Higher learning rates**: 10-100× larger without divergence
- **Less sensitivity to initialization**: Networks train from random init
- **Regularization effect**: Reduces need for Dropout
- **Deeper networks**: Essential for training 100+ layer networks

Implementation

```python
import torch.nn as nn
# Standard pattern: Conv -> BatchNorm -> ReLU
layer = nn.Sequential(
    nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
    nn.BatchNorm2d(128),  # BN has its own bias (beta)
    nn.ReLU(inplace=True)
)
```

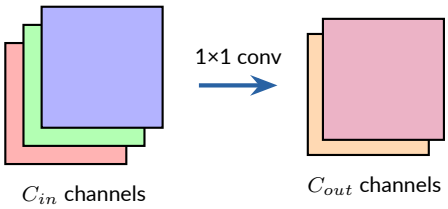
5 Network in Network: The Power of 1×1 Convolutions

Think About It

A 1×1 convolution looks at only one pixel at a time. What could it possibly learn? Is it just a pointless operation?

5.1 What 1×1 Convolutions Actually Do

A 1×1 convolution is a **per-pixel fully-connected layer** across channels:



$$y_{i,j,k} = \sum_{c=1}^{C_{in}} w_{k,c} \cdot x_{i,j,c} + b_k$$

5.2 Three Powerful Uses of 1×1 Convolutions

1. **Channel reduction (bottleneck)**: Reduce $C_{in} \to C_{out}$ before expensive 3×3 convs
2. **Channel expansion**: Increase channels to add capacity
3. **Cross-channel interaction**: Learn combinations of features without spatial mixing

Computational Savings Example:
To transform $56 \times 56 \times 256 \to 56 \times 56 \times 256$ with 3×3 conv:

$$\text{Direct: } 3^2 \times 256 \times 256 \times 56^2 = 1.85\text{B operations}$$

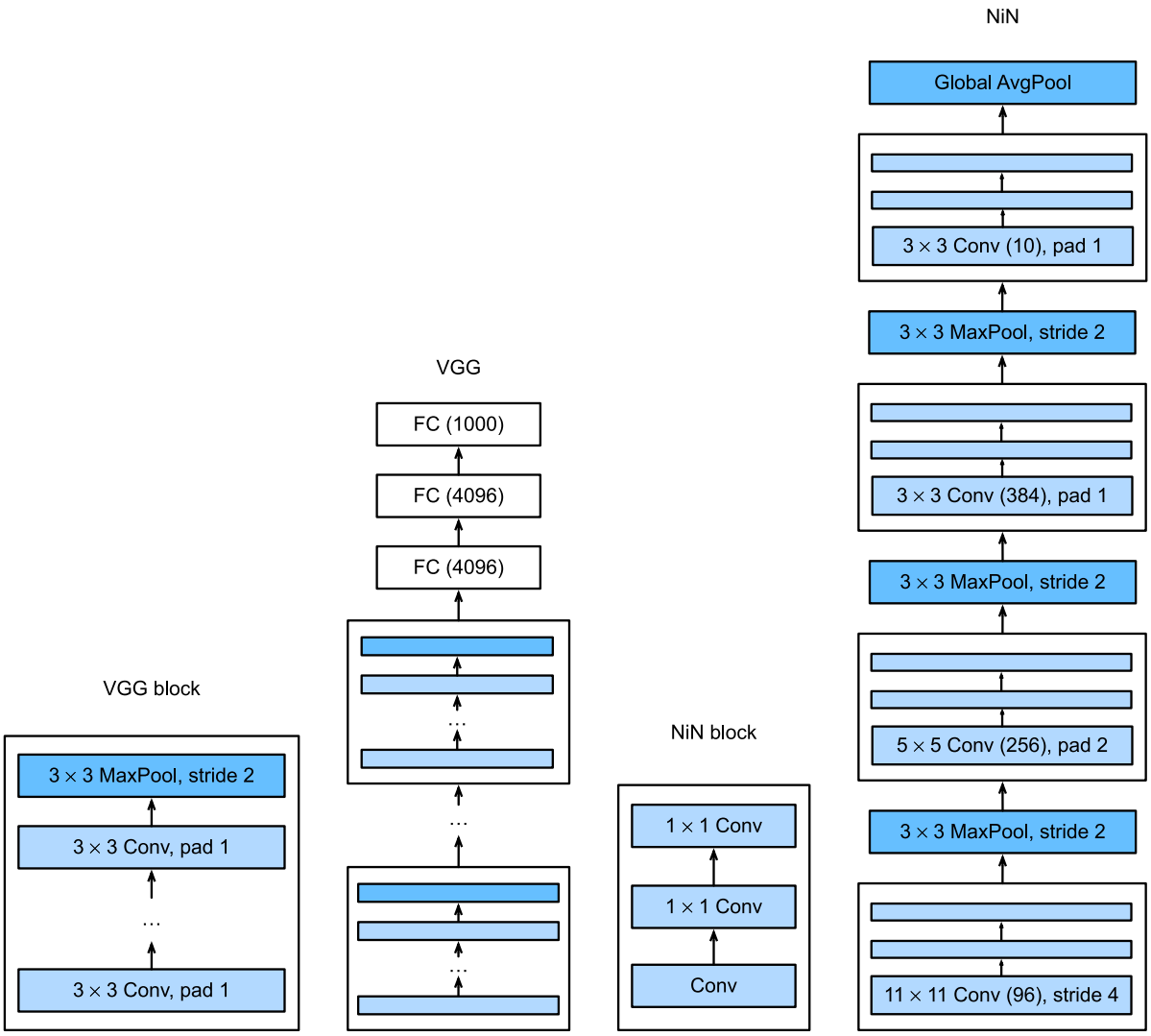
With 1×1 bottleneck ($256 \to 64 \to 256$):

$$\text{Bottleneck: } (1^2 \times 256 \times 64 + 3^2 \times 64 \times 64 + 1^2 \times 64 \times 256) \times 56^2 = 0.21\text{B}$$

Speedup: 8.8× with minimal accuracy loss!

5.3 Network in Network (NiN) Architecture

NiN introduced the concept of using 1×1 convolutions (called "mlpconv" layers) after each regular convolution, adding non-linearity within each spatial location.



Key Idea

NiN's key contribution: The mlpconv layer (1×1 conv + ReLU) acts as a "micro-network" that processes each spatial location independently. This idea became foundational for Inception and ResNet bottleneck designs.

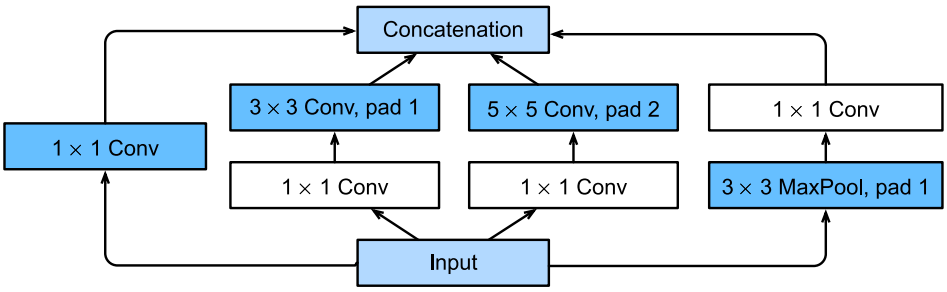
6 GoogLeNet: Inception Modules

Think About It

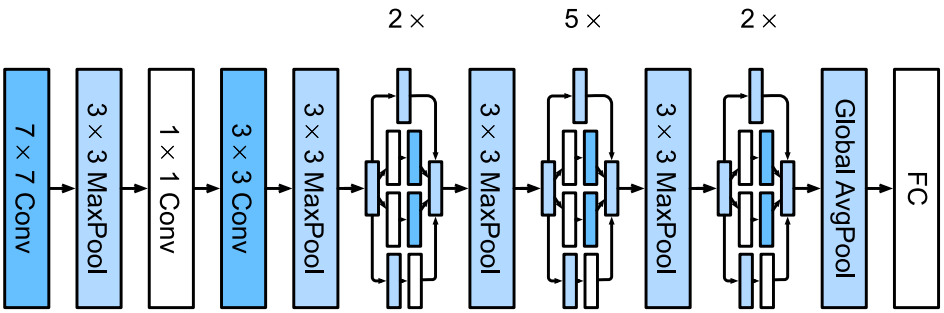
Different features exist at different scales. Should we use 1×1, 3×3, or 5×5 filters? What if we use *all of them* and let the network decide?

6.1 The Inception Module

The key insight: apply multiple filter sizes **in parallel** and concatenate results.



6.2 GoogLeNet Full Architecture



Key Idea

Inception's innovations:

- **Multi-scale processing**: 1×1, 3×3, 5×5 filters capture different feature scales
- **1×1 bottlenecks**: Reduce computation before expensive convolutions
- **Global Average Pooling**: Replaces FC layers — only 5M parameters (vs. VGG's 138M)!
- **Auxiliary classifiers**: Help gradient flow during training (removed at inference)

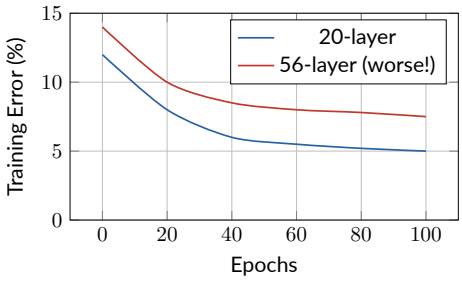
7 ResNet: The Skip Connection Revolution

Think About It

Intuitively, a 56-layer network should be at least as good as a 20-layer network (it could just learn identity for extra layers). But experiments showed **deeper networks performed worse**. Why?

7.1 The Degradation Problem

This is **not** overfitting (training error was also higher). The problem: deep networks have trouble learning identity mappings when needed.

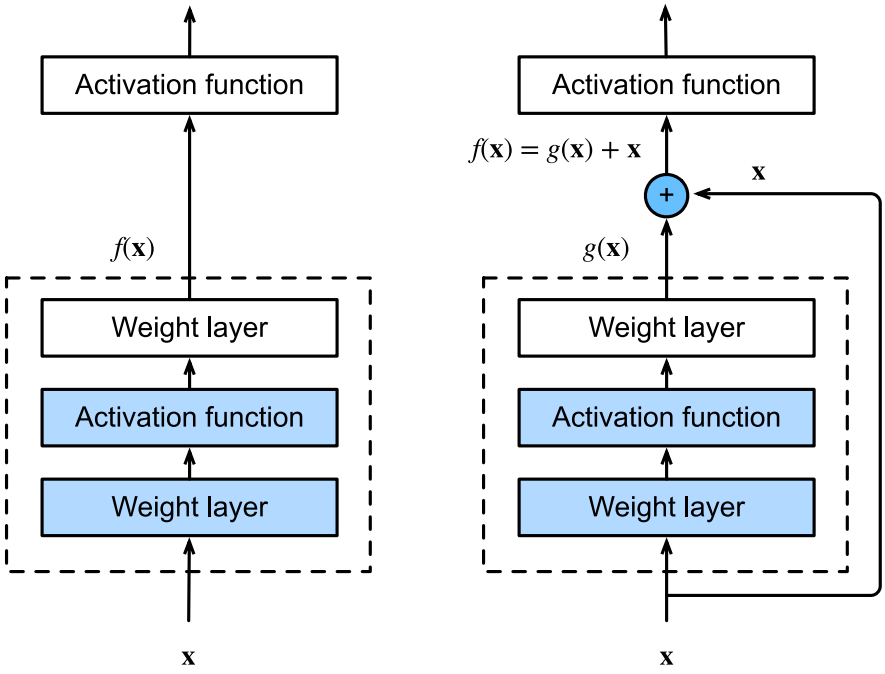


7.2 The Solution: Residual Learning

Key Insight: Instead of learning $H(x)$ directly, learn the **residual** $F(x) = H(x) - x$.
Residual Block:

$$\boxed{y = F(x, \{W_i\}) + x}$$

If identity is optimal, the network just needs to learn $F(x) = 0$ (easy!) rather than $H(x) = x$ (hard!).



7.3 Why Skip Connections Work: Gradient Flow

Gradient Analysis:
For output $y = F(x) + x$, the gradient flowing backward is:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \left(\frac{\partial F}{\partial x} + \underbrace{1}_{\text{skip}} \right)$$

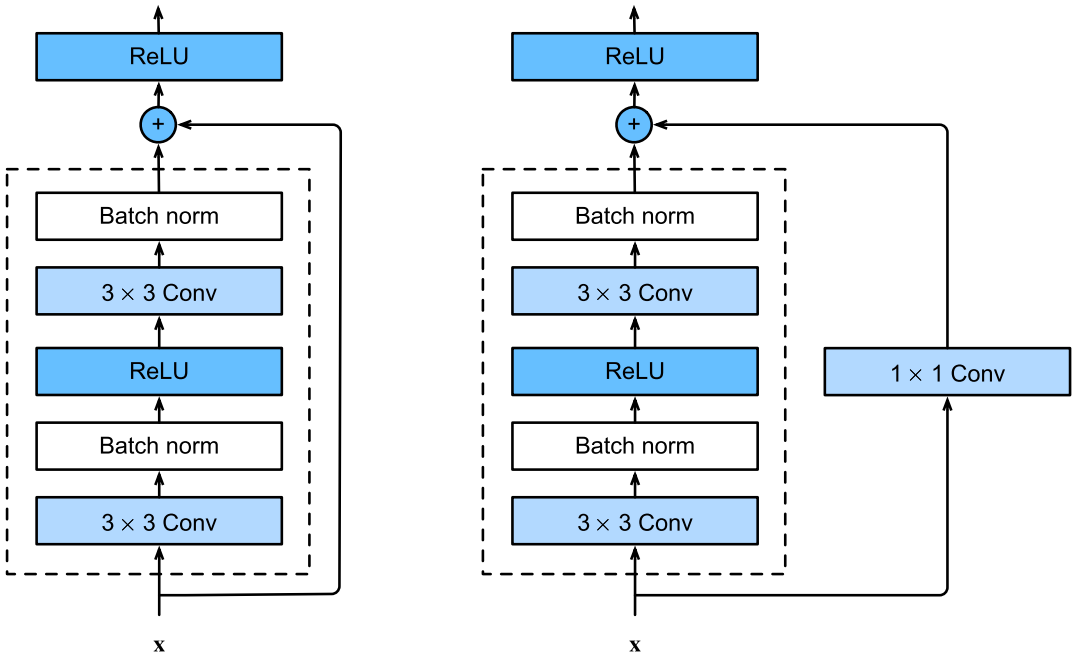
The "$+1$" ensures gradients **always flow backward**, even if $\frac{\partial F}{\partial x} \approx 0$!

Key Idea

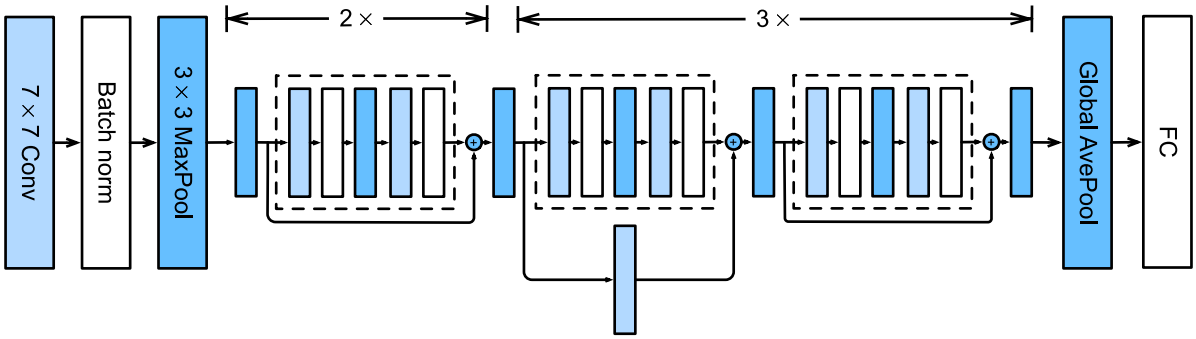
Skip connections create an **information highway**—gradients can skip problematic layers entirely. This solves vanishing gradients in very deep networks.

7.4 ResNet Bottleneck Block

For deeper networks (ResNet-50+), use a **bottleneck** design to reduce computation:



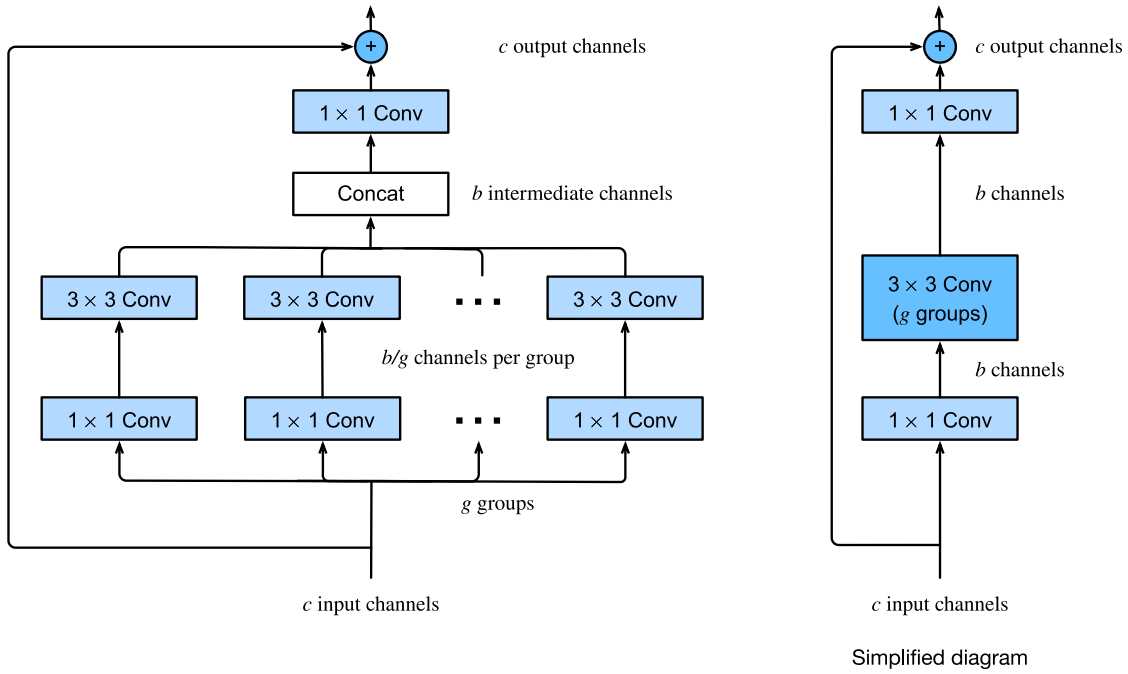
7.5 ResNet Architecture Overview



8 Beyond ResNet: ResNeXt, DenseNet, and Design Principles

8.1 ResNeXt: Aggregated Residual Transformations

ResNeXt improves upon ResNet by using **multiple parallel pathways** within each block, introducing a new dimension called **cardinality**.

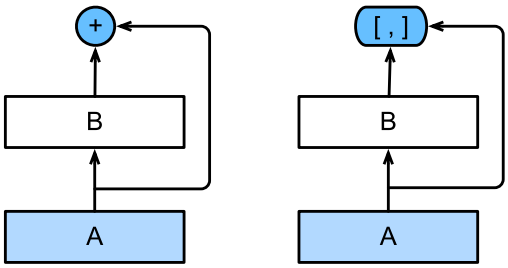


Key Idea

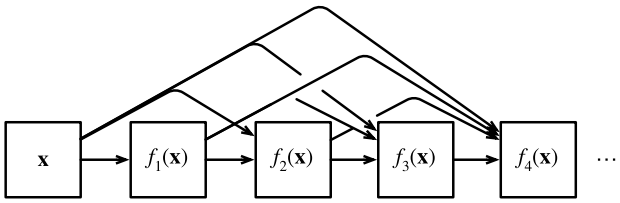
ResNeXt insight: Increasing cardinality (number of parallel paths) is more effective than increasing depth or width. ResNeXt-101 (32×4d) achieves better accuracy than ResNet-101 with similar complexity.

8.2 DenseNet: Maximum Feature Reuse

DenseNet takes skip connections to the extreme: **every layer connects to every subsequent layer**.



DenseNet Full Architecture:



Key Idea

DenseNet advantages over ResNet:

- **Better gradient flow**: Direct connections to all previous layers
- **Feature reuse**: Each layer can access all prior features
- **Fewer parameters**: DenseNet-121 (8M) vs ResNet-50 (25M) with similar accuracy
- **Implicit deep supervision**: Early layers get direct gradients from loss

8.3 Modern Design Principles

CNN Architecture Design Rules

1. **Use small filters**: 3×3 is standard; stack for larger receptive fields
2. **Use Batch Normalization**: After every conv, before activation
3. **Use skip connections**: Enable training of very deep networks
4. **Use 1×1 bottlenecks**: Reduce computation in wide networks
5. **Replace FC with Global Average Pooling**: Reduces parameters dramatically
6. **Double channels when halving resolution**: Maintains computational balance

Implementation

```python
# Modern ResNet-style block in PyTorch
class ResBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        # Skip connection with projection if dimensions change
        self.skip = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.skip = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.skip(x)  # Skip connection!
        return F.relu(out)
```

9 Summary: The Evolution of CNN Architecture

Network	Year	Depth	Params	Key Innovation
AlexNet	2012	8	60M	ReLU, Dropout, GPU training
VGG	2014	19	138M	Small 3×3 filters throughout
GoogLeNet	2014	22	5M	Inception modules, 1×1 convs
ResNet	2015	152	60M	Skip connections
DenseNet	2017	121+	8M	Dense connections

Key Takeaways

1. **Depth matters**, but only with proper techniques (BatchNorm, skip connections)
2. **Skip connections** solve the degradation problem by ensuring gradient flow
3. **1×1 convolutions** enable efficient computation and cross-channel learning
4. **Batch Normalization** stabilizes training and allows higher learning rates
5. Modern architectures use these building blocks in various combinations

Next Lecture: Transfer learning and fine-tuning pretrained models.
Practice: Implement ResNet-18 from scratch and train on CIFAR-10.